Instruction Manual: Implementing a 2D Max Pooling Layer (MaxPool2d)

1 Overview of 2D Max Pooling

A 2D max pooling layer reduces the spatial dimensions of an input tensor by taking the maximum value over a specified window (kernel) for each channel, commonly used in convolutional neural networks (CNNs) to downsample feature maps. This manual provides a step-by-step guide to implement a `MaxPool2d` layer, including forward and backward passes. The implementation is non-trainable (no learnable parameters) and does not support dilation.

1.1 Key Parameters

- **Input Tensor**: Shape $[B, C, H, W]$ (denoted X), where B is batch size, C is channels, H is height, W is width.
- **Output Tensor**: Shape $[B, C, H_{\text{out}}, W_{\text{out}}]$ (denoted Y).
- **Kernel Size**: (k_h, k_w), height and width of the pooling window.
- **Stride**: (s_h, s_w), step size for sliding the window.
- **Padding**: Options: "valid" (no padding) or custom $(p_{\text{left}}, p_{\text{right}}, p_{\text{top}}, p_{\text{bottom}})$.
- **Padding Mode**: "zero" (zero-padding).
- **Layer Name**: String identifier (default "MaxPool2D").

1.2 Assumptions

- Tensors are 4D with dimensions $[B, C, H, W]$.
- No dilation is supported (fixed at $(1, 1)$).
- The layer is non-trainable (no weights or biases).

2 Mathematical Formulation

2.1 Forward Pass

The forward pass computes Y by taking the maximum value over a kernel window in X for each position. For $Y[b, c, oh, ow]$:

$$Y[b, c, oh, ow] = \max_{\substack{0 \le kh < k_h \\ 0 \le kw < k_w}} X[b, c, ih + kh, iw + kw] \tag{1}$$

Where:

- $ih = oh \cdot s_h - p_{\text{top}}$
- $iw = ow \cdot s_w - p_{\text{left}}$

The operation is applied independently for each batch b and channel c. During the forward pass, the indices of the maximum values are cached for use in the backward pass.

2.1.1 Output Dimensions

$$H_{\text{out}} = \left\lfloor \frac{H + p_{\text{top}} + p_{\text{bottom}} - k_h}{s_h} \right\rfloor + 1 \tag{2}$$

$$W_{\text{out}} = \left\lfloor \frac{W + p_{\text{left}} + p_{\text{right}} - k_w}{s_w} \right\rfloor + 1 \tag{3}$$

2.2 Backward Pass

The backward pass computes the gradient $\frac{\partial L}{\partial X}$ given $\frac{\partial L}{\partial Y}$ (denoted

2.3 Class Structure

The `MaxPool2d` class includes:

- **Attributes**: Channels, kernel size, stride, padding, layer name, input/output caches, max indices cache.

- **Methods**: Constructor, `forward`, `backward`, helpers (`pad`$_i nput$,

2.4 Constructor

Algorithm 1 MaxPool2d Constructor

functionMaxPool2dkernel$_s ize$ =(2,2), $stride$ =(2,2), $padding$ = "$valid$", $num_p adding$ =(0,0,0,0), $padding_m ode$ = "$zero$", $layer_n ame$ = "$MaxPool2D$" **if** $kernel_s ize$[0]≤ 0 **or** kernel$_s ize$[1]≤ 0 **then**
    throw "Kernel size must be positive"

    if stride[0] ≤ 0 **or** stride[1] ≤ 0 **then**
        throw "Stride must be positive"
    end if
    if padding ∉ ["valid"] **then**
        throw "Only valid padding is supported"
    end if
    if padding$_m ode$≠ "zero" **then**
        throw "Only zero-padding is supported"
    end if
    if num$_p adding$[0, 1, 2, 3]¡ 0 **then** **throw**"$Paddingvaluesmustbenon - negative$"
    function
        this.kernel$_s ize$← kernel$_s ize$ $this.stride$← stride
        this.padding ← padding
        this.num$_p adding$← num$_p adding$ $this.padding_m ode$← padding$_m ode$
        this.layer$_n ame$← layer$_n ame$**or**"$MaxPool2D$" **end function** = 0

2.5 Padding

Algorithm 2 Pad Input

functionpad$_i nputX$ $\text{p}_{\text{left}}, p_{\text{right}}, p_{\text{top}}, p_{\text{bottom}} \leftarrow \text{num}_p adding$
    function$p_{\text{left}} = 0$ **and** $p_{\text{right}} = 0$ **and** $p_{\text{top}} = 0$ **and** $p_{\text{bottom}} = 0$
        return X

        $B, C, H, W \leftarrow$ X.shape
        padded$_X \leftarrow$ zeros(shape=[B, C, $H + p_{\text{top}} + p_{\text{bottom}}, W + p_{\text{left}} + p_{\text{right}}$])
        padded$_X$[:, :,$\text{p}_{\text{top}} : (H + p_{\text{top}}), p_{\text{left}} : (W + p_{\text{left}})] \leftarrow$ X
    return padded$_X$ **end function**

2.6 Output Dimensions

Algorithm 3 Initialize Output

functioninit$_o utput X$ B, C, H, W ← X.shape
$k_h, k_w \leftarrow$ kernel$_s ize$ $s_h, s_w \leftarrow$ stride
$p_{\text{top}}, p_{\text{bottom}}, p_{\text{left}}, p_{\text{right}} \leftarrow$ num$_p adding$ $out_h \leftarrow \lfloor (H + p_{\text{top}} + p_{\text{bottom}} - k_h)/s_h \rfloor + 1$
out$_w \leftarrow \lfloor (W + p_{\text{left}} + p_{\text{right}} - k_w)/s_w \rfloor + 1$
if out$_h \leq 0$ **or** out$_w \leq 0$ **then**
throw "Invalid output dimensions"
end if
output ← zeros(shape=[B, C, out$_h, out_w$]) **return**out_h, out_w
function

2.7 Forward Pass

Algorithm 4 Forward Pass

functionforwardX
in$_c ache \leftarrow$ X
$B, C, H, W \leftarrow$ X.shape
input$_p added \leftarrow$ pad$_i nput(X)$ $out_h, out_w \leftarrow$ init$_o utput(X)$
output ← zeros(shape=[B, C, out$_h, out_w$]) $max_i ndices \leftarrow$ zeros(shape=[B, C, out$_h, out_w$, 2])
b in 0 to $B - 1$
for c in 0 to $C - 1$ **do**
for oh in 0 to out$_h$-1 **do** **for** $owin0toout_w$-1 **do**
for i **do**h$_s tart \leftarrow$ oh $\cdot s_h - p_{\text{top}}$
iw$_s tart \leftarrow$ ow $\cdot s_w - p_{\text{left}}$
max$_v al \leftarrow -\infty$
max$_i dx \leftarrow$ [0, 0]
for kh in 0 to $k_h - 1$ **do**
for kw in 0 to $k_w - 1$ **do**
ih ← ih$_s tart + kh$ $iw \leftarrow$ iw$_s tart + kw$
ih ≥ 0 **and** ih $< H + p_{\text{top}} + p_{\text{bottom}}$ **and** iw ≥ 0 **and** iw $< W + p_{\text{left}} + p_{\text{right}}$
val ← input$_p added[b, c, ih, iw]$ **if** $val ¿ max_v al$ **then**
max$_v al \leftarrow$ val
max$_i dx \leftarrow$ [kh, kw]
end for
end for
output[b, c, oh, ow] ← max$_v al$ $max_i ndices[b, c, oh, ow] \leftarrow$ max$_i dx$
this.max$_i ndices \leftarrow$ max$_i ndices$ **return**$output$
=0

2.8 Backward Pass

3 Example Usage

```
pool = MaxPool2d(
```

Algorithm 5 Backward Pass

functionbackwardgrad$_out$ **if** $grad_out.shape \neq$ [B, C, out$_h$, out_w] **then**
functionthrow "Gradient output dimensions mismatch"

$B, C, H, W \leftarrow$ in$_cache.shape$ $grad_input \leftarrow$ zeros(shape=[B, C, H, W])
input$_padded \leftarrow$ pad$_input(in_cache)$ **for** $bin0to$B-1 **do**
c in 0 to $C - 1$
for oh in 0 to out$_h$-1 **do** **for** $owin0toout_w$-1 **do**
for i **do**h$_start \leftarrow$ oh $\cdot s_h - p_{\text{top}}$
iw$_start \leftarrow$ ow $\cdot s_w - p_{\text{left}}$
[kh, kw] $\leftarrow \max_indices[b, c, oh, ow]$ $ih \leftarrow$ ih$_start + kh$
iw $\leftarrow$ iw$_start + kw$ **if** $ih \geq 0$ **and** ih $< H + p_{\text{top}} + p_{\text{bottom}}$ **and** iw ≥ 0 **and** iw $< W + p_{\text{left}} + p_{\text{right}}$ **then**
grad$_input[b, c, ih - p_{\text{top}}, iw - p_{\text{left}}] \leftarrow$ grad$_out[b, c, oh, ow]$

return grad$_input$ $= 0$

```
    kernel_size=(2,2), stride=(2,2), padding="valid",
    num_padding=(0,0,0,0), layer_name="MaxPool2D"
)
X = random_tensor(shape=[batch_size, 3, 28, 28])
Y = pool.forward(X)
grad_out = random_tensor(shape=Y.shape)
grad_input = pool.backward(grad_out)
```

4 Error Handling

- Validate input dimensions and parameters.
- Check for non-integer or negative output dimensions.
- Ensure gradient shape matches input shape.

5 Optimization Tips

- Use vectorized operations for max computations if supported by the language.
- Cache input and max indices to avoid redundant computations.
- Parallelize loops for large inputs.